SIMATS ENGINEERING

ASSESSMENT TOOL 2 : Scenario-Based (Set 3)

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 25**
Question Count: 5

Code of Conduct

I K. Ragavi (192524364) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K. Ragavi

Assessment Tasks

Scenario-Based Questions

Question 1

A manufacturing company plans to migrate its legacy enterprise applications and databases from an on-premises data center to the cloud. The organization wants to minimize downtime while ensuring application performance and security.

- Design a cloud migration strategy suitable for the organization.
- Explain how different migration approaches (Rehost, Refactor, Replatform, Repurchase, Retire, Retain) can be applied.
- Discuss methods to validate application performance after migration.
- Recommend strategies to reduce migration risks and downtime.

Answer

1. Understanding of the Scenario

The company runs legacy ERP, production and database systems that support the factory floor. Any outage stops production, so the migration must be phased, tested and reversible, while preserving performance and security.

2. Key Risks and Issues

- Downtime during cutover of databases and ERP.
- Hidden dependencies between applications, and legacy software that is incompatible with the cloud.

- Performance drop due to network latency or wrong sizing.
- Security and compliance gaps, and data loss or corruption during transfer.
- Cost overruns and skills shortage.

3. Migration Strategy

- **Assess:** Discover all applications, dependencies, data volumes and compliance needs (AWS Migration Hub, Azure Migrate); classify applications by criticality.
- **Plan:** Group applications into waves, starting with low-risk systems. Build a landing zone (network, identity, security, governance) and a dedicated link (Direct Connect/ExpressRoute).
- **Migrate:** Continuously replicate servers and databases (e.g., DMS), test, then perform a short final cutover.
- **Optimize:** Right-size, automate and modernize after the move.

4. Applying the Six Migration Approaches

Approach	Application in this scenario
Rehost (lift and shift)	Move legacy ERP and file servers as VMs with no code change; fastest, lowest risk
Replatform	Move databases to managed services (e.g., Amazon RDS) with minimal change to gain backups and HA
Refactor	Re-architect production-planning and IoT/analytics applications into microservices or serverless for scalability
Repurchase	Replace custom CRM, HR or procurement software with SaaS
Retire	Decommission unused or duplicate applications to cut cost and risk
Retain	Keep shop-floor systems bound to hardware or regulation on premises (hybrid)

5. Validating Performance After Migration

- Compare with a pre-migration baseline of response time, throughput, CPU, memory and I/O.
- Run functional, integration, load and stress tests, plus user acceptance testing.
- Monitor real traffic with APM tools (CloudWatch, Azure Monitor, Datadog); verify data integrity with checksums and row counts; check security configuration.

6. Reducing Risks and Downtime

- Pilot migration and rehearsals with detailed runbooks.
- Continuous replication with a short final sync in a maintenance window.
- Blue-green or phased cutover with a tested rollback plan.
- Backups before migration, training, and clear communication with stakeholders.

7. Justification

A phased, wave-based approach using the right "R" for each application balances speed, cost and risk; continuous replication and rollback plans directly address the downtime requirement.

Question 2

A smart city project deploys thousands of IoT sensors to monitor traffic, air quality, energy consumption, and public safety. The collected data must be processed and analyzed in real time for decision-making.

- Design a cloud-based architecture for IoT data collection and analytics.
- Explain how message brokers, stream processing, and cloud storage are integrated.
- Discuss methods for handling high-volume, high-velocity sensor data.
- Recommend techniques to ensure scalability, reliability, and security.

Answer

1. Understanding of the Scenario

Thousands of sensors continuously send data in different formats. Decisions such as changing traffic signals or issuing pollution alerts depend on results within seconds, while historical data is needed for planning.

2. Key Risks and Issues

- Very high data volume and velocity, with bursts.
- Network latency and unreliable connectivity from devices.
- Insecure or spoofed devices; privacy of citizen data.
- Message loss or duplication, and storage cost growth.

3. IoT Architecture

- **Device layer:** Sensors and cameras send data via MQTT/CoAP/HTTPS, usually through gateways.
- **Edge layer:** Local filtering, aggregation and quick analytics to cut latency and bandwidth.
- **Cloud ingestion:** IoT hub/core (AWS IoT Core, Azure IoT Hub) authenticates devices and routes messages.
- **Processing and storage:** Stream processing for real-time rules, a time-series database for recent data, and a data lake for long-term analytics and ML.
- **Applications:** Dashboards, alerts, traffic control and city operation centers.

4. Integration of Message Broker, Stream Processing and Storage

Devices publish messages to a broker (MQTT broker/Kafka), which decouples producers from consumers and buffers bursts. Stream processors (Flink, Spark Streaming, Kinesis Analytics) subscribe, clean and enrich the data and detect events, such as pollution above a threshold, in real time. The same stream is written to hot storage (time-series DB) for dashboards and to cold object storage for history and machine learning.

5. Handling High-Volume, High-Velocity Data

- Partitioned topics for parallel processing and buffering of bursts.
- Edge filtering, downsampling and compression; send only changes or aggregates.
- Lightweight protocols and binary formats; batch writes.
- **Tiered retention:** raw data for days, aggregates for years.

6. Scalability, Reliability and Security

- **Scalability and reliability:** Autoscaling consumers, multi-AZ replicated brokers, retries and dead-letter queues, offline buffering on devices.
- **Security:** Unique device identity with X.509 certificates, TLS encryption, secure boot, signed over-the-air updates, least-privilege policies, network segmentation, anomaly detection for rogue devices, and anonymization of citizen data.

7. Justification

Edge processing and a broker-based streaming pipeline give low latency and absorb bursts, while tiered storage keeps cost under control; device identity and encryption address the main trust risks.

Question 3

A global e-commerce company wants executives to monitor sales performance, customer behaviour, and regional trends through interactive dashboards. The solution must support real-time analytics and historical reporting.

- Design a cloud-based business intelligence architecture.
- Explain how data visualization tools integrate with cloud data warehouses.
- Discuss techniques for optimizing dashboard performance with large datasets.
- Recommend security measures to protect business reports and sensitive data.

Answer

1. Understanding of the Scenario

Executives need fast, interactive and trustworthy dashboards that combine live data (current sales) with years of history across regions, and the reports contain sensitive commercial and customer information.

2. Key Risks and Issues

- Slow dashboards on huge datasets; inconsistent metric definitions between teams.
- Stale data if pipelines fail; high query cost.
- Data leakage and unauthorized access to regional or customer data.

3. BI Architecture

- **Sources:** Orders, payments, clickstream, CRM and inventory systems.
- **Ingestion:** Batch ETL/ELT for history and streaming (Kafka/Kinesis) for real-time events.
- **Storage:** Data lake plus a cloud data warehouse (Snowflake, BigQuery, Redshift) with a semantic layer defining standard metrics such as revenue and conversion rate.
- **Visualization:** Tableau, Power BI, Looker or QuickSight delivering interactive dashboards on web and mobile.

4. Integration of Visualization Tools with Data Warehouses

BI tools connect through native connectors or JDBC/ODBC. In live (direct query) mode the dashboard sends SQL to the warehouse for up-to-date results; in extract/import mode data is cached in the BI engine for speed. A semantic layer, scheduled refreshes and single sign-on tie user identity to warehouse permissions.

5. Optimizing Dashboard Performance

- Pre-aggregated tables, materialized views and summary marts instead of raw data.
- Partitioning and clustering of tables; result caching and in-memory extracts (Tableau Hyper, SPICE).
- Limit visuals, filters and returned rows per page; incremental refresh and filter pushdown.
- **Hot/cold split:** streaming data for recent hours, aggregates for history; scale warehouse compute at peak times.

6. Security Measures

- SSO with MFA and role-based access; row-level and column-level security so each region sees only its own data.
- Encryption at rest and in transit, masking of sensitive fields, and expiring share links.
- Audit logs of report access, data loss prevention, and governance of certified datasets.

7. Justification

Combining streaming and batch with a warehouse and semantic layer gives consistent real-time and historical views; pre-aggregation and caching keep dashboards fast, and row-level security protects the data.

Question 4

An online gaming company experiences fluctuating workloads throughout the day. During peak hours, users report high latency and reduced application performance.

- Design a cloud monitoring and performance optimization solution.
- Explain how cloud monitoring tools, metrics, logs, and tracing help identify performance bottlenecks.
- Discuss strategies for resource optimization and capacity planning.
- Recommend methods to improve application availability while controlling operational costs.

Answer

1. Understanding of the Scenario

Demand varies strongly by time of day. At peak, fixed capacity or an overloaded component (servers, database, network) causes high latency, which hurts the gaming experience and player retention.

2. Key Risks and Issues

- Insufficient or slow-to-scale capacity at peak; database or matchmaking bottlenecks.
- Players far from game servers; network congestion.
- Lack of visibility into the cause; overspending on idle capacity off-peak.

3. Monitoring and Optimization Design

- Instrument game servers, matchmaking, databases and networks with agents/SDKs sending metrics, logs and traces to a central platform (CloudWatch, Prometheus-Grafana, Datadog).
- Dashboards for player latency, tick rate, concurrent users, error rate and resource usage; alerts and SLOs (e.g., 95% of players under 50 ms).
- Real-user and synthetic monitoring from player regions.
- Autoscaling and regional game-server placement driven by the findings.

4. How Metrics, Logs and Tracing Identify Bottlenecks

Signal	Purpose	Example finding
Metrics	Numeric trends over time	CPU at 95% or database connections exhausted at 8 PM
Logs	Detailed event records	Timeouts and errors in matchmaking service
Distributed tracing	Follows a request across services	Slow leaderboard query adds 300 ms to each login

Correlating all three shows where (service), why (cause) and when (peak load) latency occurs.

5. Resource Optimization and Capacity Planning

- Right-size instances and remove idle resources; cache with Redis; use database read replicas or sharding.
- Forecast daily, weekly and seasonal peaks (releases, events) from historical data and load test before launches.
- Predictive and scheduled autoscaling with 20-30% headroom.
- Route players to the nearest region and deliver assets via CDN.

6. Availability and Cost Control

- Multi-zone or multi-region deployment with health checks and automatic replacement of failed servers.
- Reserved instances for baseline, on-demand for peaks, spot instances for non-critical jobs.

- Scale in off-peak, use budgets and cost alerts, and test resilience with failure drills.

7. Justification

Observability finds the real bottleneck instead of guessing, and a mixed purchasing and scaling model keeps performance high at peak without paying for idle capacity.

Question 5

A financial institution stores petabytes of customer transaction data for regulatory compliance and business analytics. The organization must retain historical records while managing storage costs efficiently.

- Design a cloud-based data lifecycle management strategy.
- Explain how data classification, retention policies, and archival storage should be implemented.
- Discuss the importance of governance, auditing, and regulatory compliance in big data environments.
- Recommend techniques for balancing data accessibility, security, and cost optimization.

Answer

1. Understanding of the Scenario

Petabytes of regulated data must be kept for years, remain retrievable for audits and analytics, and stay secure, yet storing everything on expensive fast storage is not sustainable.

2. Key Risks and Issues

- Rapidly growing storage cost; non-compliance fines if records are deleted early or kept too long.
- Data breach or tampering of sensitive records.
- Slow or costly retrieval from archives; lack of ownership and visibility.

3. Data Lifecycle Strategy

Stage	Age (example)	Storage tier	Access
Hot	0–90 days	Standard storage / warehouse	Immediate, frequent
Warm	3–12 months	Infrequent-access tier	Milliseconds, lower cost
Cold / archive	1–7+ years	Glacier / archive tier with WORM lock	Minutes to hours
Deletion	After retention expires	Secure deletion	Documented and approved

Lifecycle rules move data automatically between tiers, and a catalog keeps archived data discoverable.

4. Classification, Retention and Archival

- **Classification:** Label data as public, internal, confidential or restricted (PII, account data) using automated discovery (Macie, Purview) and tags; this drives encryption, access and retention.
- **Retention policies:** Set periods per regulation (many financial records 5–10 years), applied automatically; legal hold suspends deletion when required.
- **Archival:** Immutable WORM storage (object lock), compression and metadata indexes, periodic restore tests, and secure, documented deletion at the end of retention.

5. Governance, Auditing and Compliance

Regulations such as PCI DSS, SOX, GDPR and banking laws require records to be complete, protected and retrievable. Governance assigns ownership and policies; auditing (access logs, lineage) proves who accessed what and when; compliance monitoring detects violations early. Together they avoid fines, support investigations and maintain customer trust, and data-residency rules must also be respected.

6. Balancing Accessibility, Security and Cost

- Automated and intelligent tiering, compression, columnar formats, and removal of duplicate data.

- Encryption with managed keys, role-based access, MFA and immutable backups.
- Keep summaries or indexes in hot storage so analytics rarely need to restore archives.
- Monitor growth and cost with budgets, tagging and periodic reviews.

7. Justification

Tiering by age and classification keeps compliance and security intact while moving rarely used data to cheap storage, giving the best trade-off between access speed and cost.

Scenario-Based Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented